Turtle Graphics Project, Spring 2026

Tong Yi

Contents

1 CopyRight Warning

1. This is copyrighted materials; you are not allowed to upload to the Internet.
2. Our project is different from similar products in Internet.
   (a) Ask help only from teaching staff of this course.
   (b) Use solutions from artificial intelligence like ChatGPT or online tutoring websites like, but not limited to, chegg.com, violates academic integrity and is not allowed.

2 Example of Separating Declaration, Definition, and Test Code: FuelTank Class

For robust software engineering practices, especially in large projects, the standard C++ approach is to divide the code into three distinct files:

Header File (.h or .hpp): Contains the declaration of the class (the class blueprint, member variables, and function prototypes).

Source File (.cpp): Contains the definitions (implementation logic) of the class's member functions.

Test File (.cpp): Contains the main() function and the code dedicated to testing the functionality of the class.

This separation improves compile times (only changed implementation files need recompilation), enhances code organization, and facilitates code reuse across different parts of a project.

Code link: https://onlinegdb.com/MzXcjYT2S-.

In online compilers like onlinegdb, the primary source file containing the `main` function must be named `main.cpp`. When working on your local computer, you have more flexibility and can name the test file something more descriptive, such as `TestFuelTank.cpp`.

2.1 FuelTank.hpp

```
#ifndef FUEL_TANK_H
#define FUEL_TANK_H
class FuelTank {
public:
    FuelTank();
    FuelTank(int startLevel);
    int getGasLevel() const;
    void fillGas(int numGallons);
    void sendGas(int numGallons);

private:
    int currGasLevel;
};
#endif
```

2.2 FuelTank.cpp

```
#include "FuelTank.hpp"

FuelTank::FuelTank() {
   //int currGasLevel = 0; //wrong, with int before currGasLevel, the currGasLevel is
       a local variable
   //for default constructor,
   //that statement does not initialize data member currGasLevel
   currGasLevel = 0;
}

FuelTank::FuelTank(int startGasLevel) {
    if (startGasLevel >= 0) {
       currGasLevel = startGasLevel;
    }
    else {
       currGasLevel = 0;
    }
}

int FuelTank::getGasLevel() const {
    return currGasLevel;
}

void FuelTank::sendGas(int gallons) {
    if (currGasLevel >= gallons) {
       currGasLevel -= gallons;
```

```
    }
}

void FuelTank::fillGas(int gallons) {
    currGasLevel += gallons;
}
```

2.3 TestFuelTank.cpp

```
//Sample output:
//gas level of an empty fuel tank is 0
//add 5 gallons to tank
//send 3 gallons out from tank
//current gas level is 2

#include "FuelTank.hpp"
#include <iostream>

int main() {
    //TODO: Call default constructor of FuelTank class to instantiate an object called
        tank
    //related: call default constructor of string class to instantiate an empty string
        object called str.
    FuelTank tank(0); //same as FuelTank tank; //same as FuelTank tank; //same as
       FuelTank tank; //same as FuelTank tank;
    //similar to
    //string str; //str is an empty string

    //TODO: print out the current gas level
    std::cout << "gas level of an empty fuel tank is "
        << tank.getGasLevel() << std::endl; //fill in ... part

    //TODO: write a statement to add 5 gallons of gas to tank
    //related: call substr method of str object to get a substring
    std::cout << "add 5 gallons to tank" << std::endl;
    tank.fillGas(5);

    std::cout << "send 3 gallons out from tank" << std::endl;

    //TODO: write a statement to send 3 gallons of gas from tank
    tank.sendGas(3);

    std::cout << "current gas level is ";
    //TODO: write a statement to find out and print the current gas level of tank
    //related: find out the length of a string object str
```

```
    std::cout << tank.getGasLevel() << std::endl;

    //WRONG: currGasLevel is a private data member and cannot be accessed directly
       from a function outside the class where currGasLevel is defined
    //std::cout << tank2.currGasLevel << std::endl;
    return 0;
}
```

To run the code,

```
g++ -o run FuelTank.cpp TestFuelTank.cpp
./run
```

3 Overview of Turtle Graphics Project

This project involves three classes: Floor, Turtle, and TurtleGraphics.

- Floor is a square-shape grid with characters '*' and space character ' '. There are operations like `clear`, `mark`, `at` to operate on the grid.
- Turtle lets a turtle lift up or put down a pen, turning left or right, and move.
- TurtleGraphics lets a turtle to draw character '*' on the floor by turning, moving, and lifting up or putting down a pen properly.

To run the project, we do the following.

1. Build a directory in your computer by entering the following command with return key in a terminal.

   ```
   mkdir TurtleGraphics
   ```

2. Move to the directory by pressing the following command with return key in a terminal.

   ```
   cd TurtleGraphics
   ```

3. Download the following files and save in the above directory.
4. You implement Floor, Turtle, and TurtleGraphics classes. Then test TurtleGraphic class. That is,
   (a) Define Floor.cpp in Tasks A
   (b) Define Turtle.cpp in Task B.
   (c) Define TurtleGraphics.cpp in Task C.
   (d) Define TurtleGraphicsTest.cpp in Task D.

4 Task A: Implement Floor class

4.1 Download Floor.hpp

Download link: **Floor.hpp**

Its contents are as follows.

```cpp
#ifndef FLOOR_HPP //Floor_HPP is ok, but not as good as FLOOR_HPP since the former mixes small and big case letters
#define FLOOR_HPP

#include <vector>
#include <string>

/**
 * @class Floor
 * @brief Manages a 2D character grid with specific labeling and safety checks.
 */
class Floor {
public:
    /**
     * @brief Default constructor.
     * Initializes a default 20 x 20 grid filled with spaces (' ').
     */
    Floor();

    /**
     * @brief Parameterized constructor.
     * @param size The desired dimension for a square grid.
     * @note If size < 10, a default 20x20 grid is created instead.
     */
    Floor(int size);

    /** @return The number of rows in the grid. */
    int getNumRows() const;

    /** @return The number of columns in the grid. */
    int getNumCols() const;

    /**
     * @brief Places a character at the specified coordinates.
     * @param row The row index from 0 to getNumRows() - 1.
     * @param col The column index from 0 to getNumCols() - 1.
     * @param ch The character to place.
     * @note If indices are out of bounds, the operation is silently ignored.
     */
    void mark(int row, int col, char ch);
```

```
40
41      /**
42       * @brief Retrieves the character at the specified coordinates.
43       * @param row The row index from 0 to getNumRows() - 1.
44       * @param col The column index from 0 to getNumCols() - 1.
45       * @return The character at (row, col), or the null character '\0' if out of
             bounds.
46       */
47      char at(int row, int col) const;
48
49      /**
50       * @brief Resets all cells in the grid to a space (' ').
51       */
52      void clear();
53
54      /**
55       * @brief Returns a string representation of the grid with coordinate labels.
56       *
57       * Labeling Rules:
58       * 1. Designed for grids up to 99x99.
59       * 2. The tens digit for row/column indices is displayed ONLY at multiples
60       *    of 10 (e.g., 10, 20, 30), appearing directly above or to the left.
61       * 3. The units digit is displayed for every row and column.
62       *
63       * @return A formatted string containing the labels and grid content.
64       */
65      std::string to_string() const;
66
67  private:
68      std::vector<std::vector<char>> grid;
69  };
70
71  #endif
```

4.2 Implement Floor.cpp

Read the requirement of Floor.hpp. Implement the constructors and methods in Floor.cpp.

4.3 Test Floor.cpp using FloorUnitTest.cpp

The contents of FloorUnitTest.cpp are as follows. Its purpose is to test the constructors and methods of Floor.cpp.

```
1  //File name: FloorUnitTest.cpp
2  #include <iostream>
3  #include <vector>
```

```cpp
#include <cstdlib> //srand
#include <ctime> //time
#include <sstream> //std::ostringstream
#include "Floor.hpp"
//g++ -o floor Floor.cpp FloorUnitTest.cpp
//test default constructor using
//./floor A

const int SIZE = 16;
Floor assignData(char data[][SIZE]);
void printCanvas(const Floor& canvas, const std::string& prompt);

int main(int argc, const char *argv[]) {
    if (argc != 2) {
       std::cout << "Need 'A'-'G' in command line parameter" << std::endl;
       return -1;
    }

    //unit-testing for constructors and methods of Floor class
    char type = *argv[1];
    std::string prompt;

    //IMPORTANT: If your output is inconsistent, temporarily replace srand(time(NULL))
        with srand(0).
    srand(time(NULL));

    //'A' for default constructor and
    //'B' for non-default constructor
    if (type == 'A' || type == 'B') {
      if (type == 'A') {
        prompt = "Call default constructor Floor canvas;";
        Floor canvas;
        printCanvas(canvas, prompt);
//Sample output:
//Call default constructor Floor canvas;
//num of rows: 20
//num of cols: 20
//contents of canvas is
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
```

```
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,

     }
     else if (type == 'B') {
           std::cout << "Enter size of the canvas (needs to be at least 10, otherwise,
              convert it to 20): ";
           int size;
           std::cin >> size;
           prompt = "Call non-default constructor Floor canvas(" + std::to_string(size)
               + ");";
           Floor canvas(size);
           printCanvas(canvas, prompt);

//Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 11
//Call non-default constructor Floor canvas(11);
//num of rows: 11
//num of cols: 11
//contents of canvas is
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
//

//do not take the parameter as it is,
//need to make sure that parameter size is >= 10
//Sample input/output:
//Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 5
```

```
93 //Call non-default constructor Floor canvas(5);
94 //num of rows: 20
95 //num of cols: 20
96 //contents of canvas is
97 // , , , , , , , , , , , , , , , , , , , ,
98 // , , , , , , , , , , , , , , , , , , , ,
99 // , , , , , , , , , , , , , , , , , , , ,
100 // , , , , , , , , , , , , , , , , , , , ,
101 // , , , , , , , , , , , , , , , , , , , ,
102 // , , , , , , , , , , , , , , , , , , , ,
103 // , , , , , , , , , , , , , , , , , , , ,
104 // , , , , , , , , , , , , , , , , , , , ,
105 // , , , , , , , , , , , , , , , , , , , ,
106 // , , , , , , , , , , , , , , , , , , , ,
107 // , , , , , , , , , , , , , , , , , , , ,
108 // , , , , , , , , , , , , , , , , , , , ,
109 // , , , , , , , , , , , , , , , , , , , ,
110 // , , , , , , , , , , , , , , , , , , , ,
111 // , , , , , , , , , , , , , , , , , , , ,
112 // , , , , , , , , , , , , , , , , , , , ,
113 // , , , , , , , , , , , , , , , , , , , ,
114 // , , , , , , , , , , , , , , , , , , , ,
115 // , , , , , , , , , , , , , , , , , , , ,
116 // , , , , , , , , , , , , , , , , , , , ,
117 //
118       }
119     }
120     else if (type >= 'C' && type <= 'E') {
121         char data[SIZE][SIZE];
122 
123         //set some random values for data
124         char chosenValues[] = {'*', ' '};
125         int numChosenValues = sizeof(chosenValues) / sizeof(chosenValues[0]);
126         for (int row = 0; row < SIZE; row++) {
127           for (int col = 0; col < SIZE; col++) {
128             data[row][col] = chosenValues[rand() % numChosenValues];
129           }
130         }
131         Floor canvas = assignData(data);
132 
133         if (type == 'C') {
134            //test clear method,
135            canvas.clear();
136 
137            //after calling clearing method, each element of board should be ' '
138            bool isWrong = false;
```

```
139           for (int row = 0; row < canvas.getNumRows() && !isWrong; row++) {
140             for (int col = 0; col < canvas.getNumCols() && !isWrong; col++) {
141               if (canvas.at(row, col) != ' ') {
142                 std::cout << "clear method is not correct. Each element should be a
                        space character\n";
143                 isWrong = true; //set a tag
144                 //break; //break only can break the inner loop, the outer loop still
                        runs
145               }
146             }
147           }
148
149           if (!isWrong) {
150             std::cout << "clear method is correct\n";
151           }
152           else {
153             std::cout << "clear method is not correct\n";
154           }
155         //expected output:
156         //clear method is correct
157         }
158         else if (type == 'D') {
159           //test at method
160           bool isWrong = false;
161           for (int row = 0; row < canvas.getNumRows() && !isWrong; row++) {
162               for (int col = 0; col < canvas.getNumCols() && !isWrong; col++) {
163                   if (canvas.at(row, col) != data[row][col]) {
164                     std::cout << "at method is not correct. The expected value is '" <<
                            data[row][col] << "'"
165                       << ", however, the actual value is '" << canvas.at(row, col) << "
                            '\n";
166                     isWrong = true; //set a tag
167                     //break; //break only can break the inner loop, the outer loop
                            still runs
168                   }
169               }
170           }
171
172           if (!isWrong) {
173              std::cout << "at method is correct\n";
174           }
175           else {
176             std::cout << "at method is not correct\n";
177           }
178         }
179         else if (type == 'E') {
```

```
180             //canvas.print(); //debug purpose
181
182             // build expected output manually from data[][]
183             std::ostringstream expected;
184
185             // print actual via stream
186             std::string actual = canvas.to_string();
187
188             // build expected string from data[][]
189             expected << " ";
190             for (int col = 0; col < SIZE; col++) {
191               if (col % 10 == 0 && col / 10 >= 1) {
192                 expected << col / 10;
193               }
194               else {
195                 expected << ' ';
196               }
197             }
198             expected << '\n';
199
200             expected << " ";
201             for (int col = 0; col < SIZE; col++) {
202               expected << col % 10;
203             }
204             expected << '\n';
205
206             for (int row = 0; row < SIZE; row++) {
207               expected << (row % 10 == 0 && row / 10 >= 1 ? char('0' + row / 10) : ' '
                    );
208               expected << row % 10;
209               for (int col = 0; col < SIZE; col++) {
210                 expected << data[row][col];
211               }
212               expected << '\n';
213             }
214
215             //std::cout << "'" << actual << "'\n\n";
216             //std::cout << "'" << expected.str() << "'\n";
217             if (actual == expected.str()) {
218               std::cout << "to_string method is correct\n";
219             }
220             else {
221               std::cout << "to_string method is not correct\n";
222             }
223         //expected output:
224         //to_string method is correct
```

```
225         }
226       }
227       else if (type == 'F') {
228         //test mark method of Floor class
229         Floor canvas(SIZE);
230         canvas.mark(0, 0, '*');        // top-left corner
231         canvas.mark(SIZE-1, SIZE-1, '*');       // bottom-right corner, minimum size is
              10
232         bool cornersCorrect = canvas.at(0,0) == '*' && canvas.at(SIZE-1, SIZE-1) == '*';
233
234         canvas.mark(-1, 0, '*');    // out of bounds, should do nothing
235         canvas.mark(0, SIZE, '*');  // out of bounds, should do nothing
236         canvas.mark(SIZE, 0, '*');  // out of bounds, should do nothing
237         canvas.mark(SIZE, SIZE, '*'); // out of bounds, should do nothing
238
239         // at() should return '\0' for invalid indices
240         bool outOfBoundsCorrect = canvas.at(-1,0) == '\0' &&
241           canvas.at(0,SIZE) == '\0' && canvas.at(SIZE,0) == '\0' && canvas.at(SIZE,SIZE)
                == '\0';
242
243         int row = SIZE / 2;
244         int col = SIZE / 2;
245         canvas.mark(SIZE/2, SIZE/2, '*'); // out of bounds, should do nothing
246         bool centerCorrect = canvas.at(row, col) == '*';
247
248         row = rand() % SIZE;
249         col = rand() % SIZE;
250         canvas.mark(row, col, '*'); // ok even if (row,col) overlaps (0,0) or (SIZE-1,
              SIZE-1) since they are already '*'
251
252
253         bool randomCorrect = canvas.at(row, col) == '*';
254         if (cornersCorrect && outOfBoundsCorrect && centerCorrect && randomCorrect) {
255           std::cout << "mark method is correct\n";
256         }
257         else {
258           std::cout << "mark method is not correct\n";
259         }
260
261 //sample output:
262 //mark method is correct
263       }
264       else {
265         std::cout << "Unrecognized option: " << type << std::endl;
266         return -1;
267       }
```

```
268
269     return 0;
270 }
271
272 Floor assignData(char data[][SIZE]) {
273    Floor canvas(SIZE);
274
275    for (int row = 0; row < canvas.getNumRows(); row++) {
276        for (int col = 0; col < canvas.getNumCols(); col++) {
277            canvas.mark(row, col, data[row][col]); //set board[row][col] of *canvasPtr --
                    a Floor object -- to be data[row][col]
278        }
279    }
280
281    return canvas;
282 }
283
284 void printCanvas(const Floor& canvas, const std::string& prompt) {
285    std::cout << prompt << '\n';
286    std::cout << "num of rows: " << canvas.getNumRows() << '\n';
287    std::cout << "num of cols: " << canvas.getNumCols() << '\n';
288    std::cout << "contents of canvas is\n";
289    for (int row = 0; row < canvas.getNumRows(); row++) {
290        for (int col = 0; col < canvas.getNumCols(); col++) {
291            std::cout << canvas.at(row, col) << ',';
292        }
293        std::cout << '\n';
294    }
295 }
```

Run the following commands.

```
1 g++ -o floor Floor.cpp FloorUnitTest.cpp
```

If the code compiles, you would get `floor` executable file. To test the default constructor, run

```
1 ./floor A
```

Test the rest of constructors and methods according to the comments in `FloorUnitTest.cpp`.

4.4 Submission for Task A

Submit `Floor.cpp` to gradescope.